VIT BHOPAL UNIVERSITY

VITyarthi - Build Your Own Project

TIC-TAC-TOE GAME

A Modular Two-Player Console Game in Python

PROJECT REPORT

Student Name	Ananya Banerjee
Registration Number	26BSA10021
Faculty	Sivabalan KR
Institution	VIT Bhopal University
Submission Date	September 2026

Table of Contents

1. Introduction 3

1.1 Purpose of the Project . . . 3

1.2 Technologies and Tools Used . . . 3

2. Problem Statement 3

2.1 Objectives . . . 3

2.2 Scope . . . 4

2.3 Target Users . . . 4

3. Functional Requirements 4

3.1 Input / Output Structure . . . 4

3.2 User Workflow . . . 4

4. Non-Functional Requirements 5

5. System Architecture 5

6. Design Diagrams 6

6.1 Use Case Diagram . . . 6

6.2 Workflow Diagram . . . 7

6.3 Sequence Diagram . . . 8

6.4 Component / Class Diagram . . . 9

6.5 ER Diagram / Database Design . . . 9

6.6 Board Data Design . . . 9

7. Design Decisions and Rationale 10

8. Implementation Details 10

9. Screenshots / Results 13

9.1 Scenario A: A win with invalid inputs . . . 13

9.2 Scenario B: A draw . . . 14

10. Testing Approach 14

10.1 How to run the tests . . . 14

10.2 Unit test cases (test_game.py) . . . 15

10.3 Test run output . . . 15

10.4 Manual end-to-end tests . . . 15

11. Challenges Faced 15

11.1 Known limitations . . . 16

12. Learnings and Key Takeaways 16

13. Future Enhancements 16

1. Introduction

Tic-Tac-Toe is a classic two-player game played on a 3 x 3 grid. Players take turns marking an empty cell with their symbol (X or O), and the first player to place three of their symbols in a horizontal, vertical or diagonal line wins. If all nine cells are filled and nobody has won, the game ends in a draw.

This project implements the game as a command-line application in Python. Although the game itself is simple, the project has been built the way a larger application would be: the code is divided into small modules with a single responsibility each, all user input is validated, the game rules are kept separate from the display code, and the core logic is covered by automated unit tests.

1.1 Purpose of the Project

The purpose is to apply core programming concepts (functions, lists, loops, conditionals, modular design, input validation, exception-safe input handling and unit testing) to a real, working problem, and to document the design and implementation the way a software project is documented in practice.

1.2 Technologies and Tools Used

Item	Details
Language	Python 3 (3.6 or newer)
Libraries	Python standard library only (no third-party dependencies at run time)
Testing	unittest and unittest.mock (standard library)
Interface	Command-line / terminal (text based)
Version control	Git and GitHub
Documentation tools	Graphviz and Matplotlib (diagrams), ReportLab (this PDF)

2. Problem Statement

Many beginner game programs are written as one long script in which input handling, game rules and screen output are mixed together. Such programs are hard to read, hard to test and hard to extend. Typical weaknesses are that a wrong keystroke (a letter, a number outside the board, an occupied cell) crashes the game or silently corrupts the board, and that win and draw rules are written as long repetitive if-statements that are easy to get wrong.

Problem: Design and build a reliable, easy-to-use two-player Tic-Tac-Toe game for the terminal in which invalid input never breaks the game, the rules are implemented in a clean and testable way, and the program is organised into independent modules.

2.1 Objectives

- Build a fully playable two-player Tic-Tac-Toe game that runs in any terminal.
- Detect wins (all 8 possible lines) and draws correctly after every move.
- Validate every input so that wrong entries produce a clear message and a retry instead of a crash.
- Separate the program into modules for board handling, game logic, player input and validation.
- Verify correctness with automated unit tests.
- Document the design with architecture, use case, workflow, sequence and component diagrams.

2.2 Scope

In scope: two human players on the same computer, custom player names, board display with numbered cells, move validation, win and draw detection, one game per program run. **Out of scope:** computer opponent, network play, graphical interface, persistent storage of scores.

2.3 Target Users

Students and beginners who want a simple game to play or study, and instructors who need a small but well-structured example of modular Python code.

3. Functional Requirements

The system is made of four major functional modules. Each module maps to one source file.

ID	Module	Requirement
FR-1	Player Management (player.py)	Ask both players for their names. A blank name is replaced by the defaults 'Player 1' and 'Player 2'. Leading and trailing spaces are removed. Player 1 always plays X and Player 2 plays O.
FR-2	Board Management (game_board.py)	Create an empty board of 9 cells. Display the board as a 3 x 3 grid; empty cells show their position number (1-9) so that players know what to type, and occupied cells show X or O.
FR-3	Game Logic (game_logic.py)	Place a symbol on the board; check whether a given symbol has completed any of the 8 winning lines (3 rows, 3 columns, 2 diagonals); check whether the board is full (draw).
FR-4	Input Validation (validation.py)	Repeatedly ask for a position until the player enters a whole number from 1 to 9 that refers to an empty cell. Reject text, empty input, negative numbers, numbers outside 1-9 and occupied cells with a specific error message.
FR-5	Game Controller (main.py)	Run the game loop: show the board, prompt the current player, apply the move, check for a win, then a draw, otherwise switch player. Announce the result and end the game.

3.1 Input / Output Structure

Stage	Input (from user)	Output (to screen)
Start-up	None	Banner 'TIC-TAC-TOE GAME'
Player setup	Two names (text, may be blank)	Prompts for Player 1 (X) and Player 2 (O)
Each turn	One position, integer 1-9	Current board, whose turn it is, prompt for position
Invalid input	Non-number / out of range / taken cell	Message: 'Please enter a number.', 'Number must be between 1 and 9.' or 'That spot is already taken.'
Game end	None	Final board and 'Congratulations! (name) wins!' or 'It's a draw!'

3.2 User Workflow

The user starts the program, enters two names, and then the players alternate turns. On each turn the board is shown and the current player types a cell number. The game automatically checks the result after every move and stops as soon as somebody wins or the board is full (see the workflow diagram in Section 6).

4. Non-Functional Requirements

ID	Category	Requirement	How it is achieved
NFR-1	Usability	A first-time player should be able to play without instructions.	Empty cells display their own number, prompts state the valid range (1-9) and name the current player and symbol.
NFR-2	Reliability / Error handling	Invalid input must never crash or corrupt the game.	A validation loop in get_valid_position() re-prompts until the input is a valid, free cell; each error has its own message.
NFR-3	Maintainability	Code must be easy to read, change and extend.	One responsibility per module, small functions with docstrings, winning combinations stored in a single WIN_LINES constant, consistent naming.
NFR-4	Testability	Rules and validation must be verifiable automatically.	Pure functions for game rules and mockable input() calls allow 15 automated unit tests to run in milliseconds.
NFR-5	Performance	Every move must respond instantly.	Board is a 9-element list; checking a win is at most 8 lines x 3 cells = 24 comparisons.
NFR-6	Portability	Must run on Windows, Linux and macOS without installation.	Uses only the Python standard library and plain text output.
NFR-7	Resource efficiency	Minimal memory and CPU use.	No external libraries, no files or network access, constant-size data (9 cells).

5. System Architecture

The application follows a simple layered architecture. The controller (main.py) coordinates the other modules and contains no game rules itself. Input modules talk to the players, the display module prints the board, and the logic module owns the rules. The board state is a plain list held in memory and passed to the functions that need it.

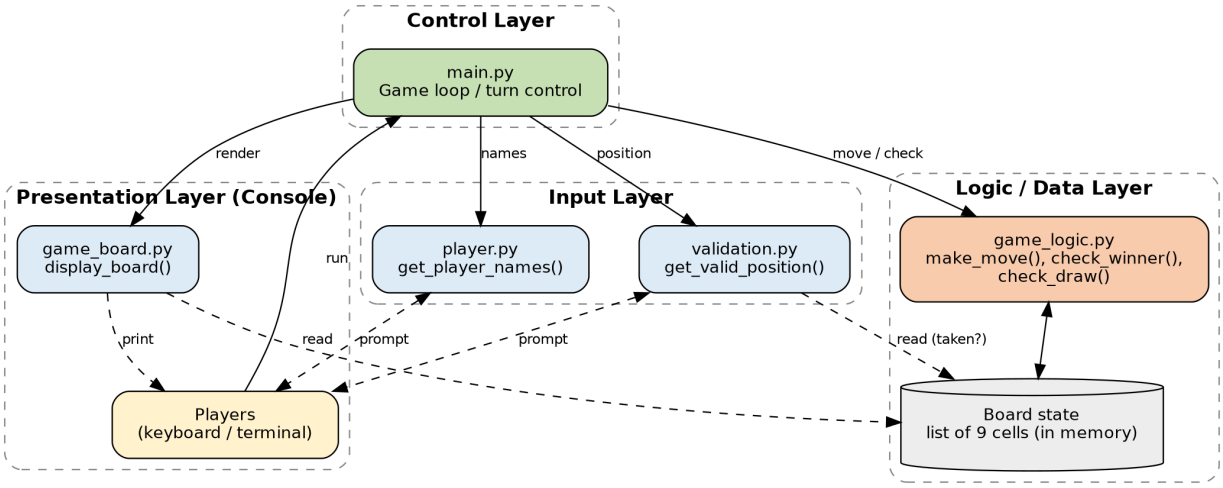


Figure 1: System architecture of the Tic-Tac-Toe application

Layer	Module(s)	Responsibility
Control	main.py	Game loop, turn order, symbol switching, announcing the result
Presentation	game_board.py (display_board)	Convert the board list into a readable 3 x 3 text grid
Input	player.py, validation.py	Read player names; read and validate a board position
Logic / Data	game_logic.py, game_board.py (create_board)	Board state, placing moves, winner and draw rules

6. Design Diagrams

6.1 Use Case Diagram

Both players interact with the system in the same way. Choosing a position always includes validating the input and placing the symbol, and placing a symbol always includes checking for a win or a draw.

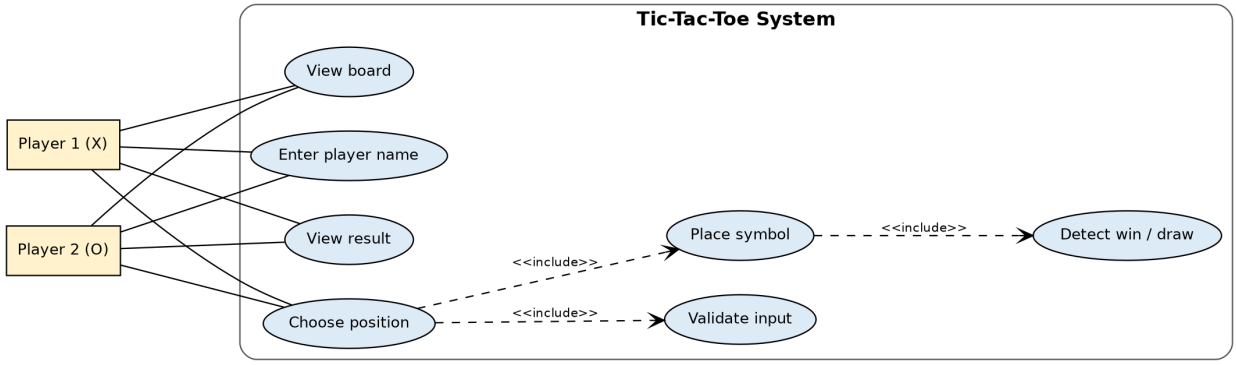


Figure 2: Use case diagram

6.2 Workflow Diagram

The flowchart shows the complete process from start-up to the end of the game, including the input validation loop and the order of the win check before the draw check.

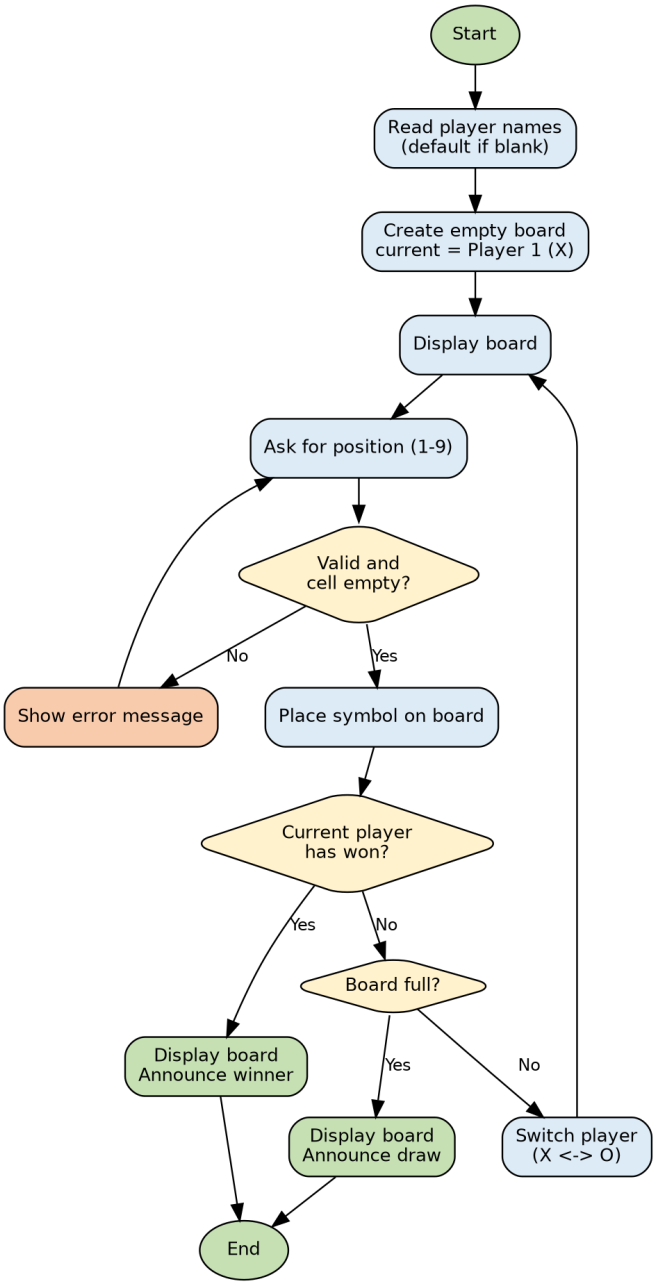


Figure 3: Workflow (flowchart) of the game

6.3 Sequence Diagram

The sequence diagram shows the message flow between the player and the modules during start-up and one turn of the game loop.

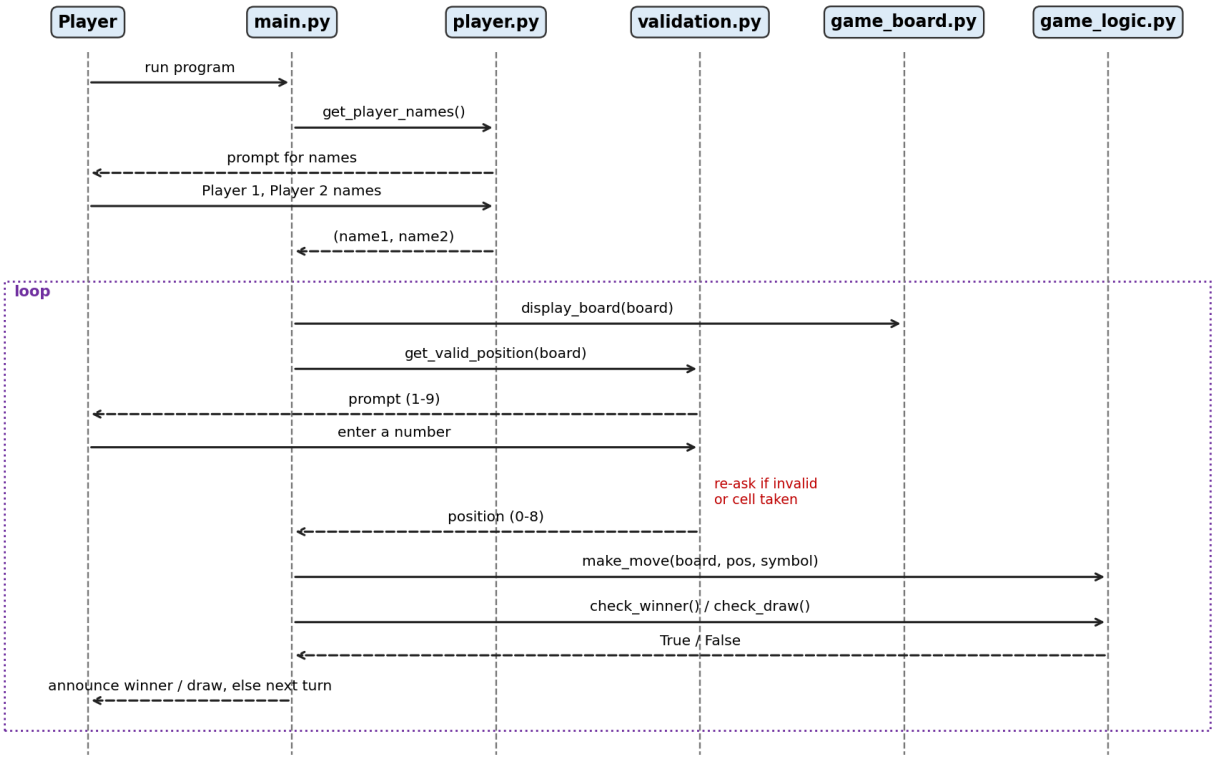


Figure 4: Sequence diagram of one game turn

6.4 Component / Class Diagram

The project uses functions grouped into modules rather than classes, so the class diagram is presented as a component diagram. Each box lists the public functions of a module; dashed arrows show that main.py imports and uses the other modules. The other modules do not depend on each other, which keeps coupling low.

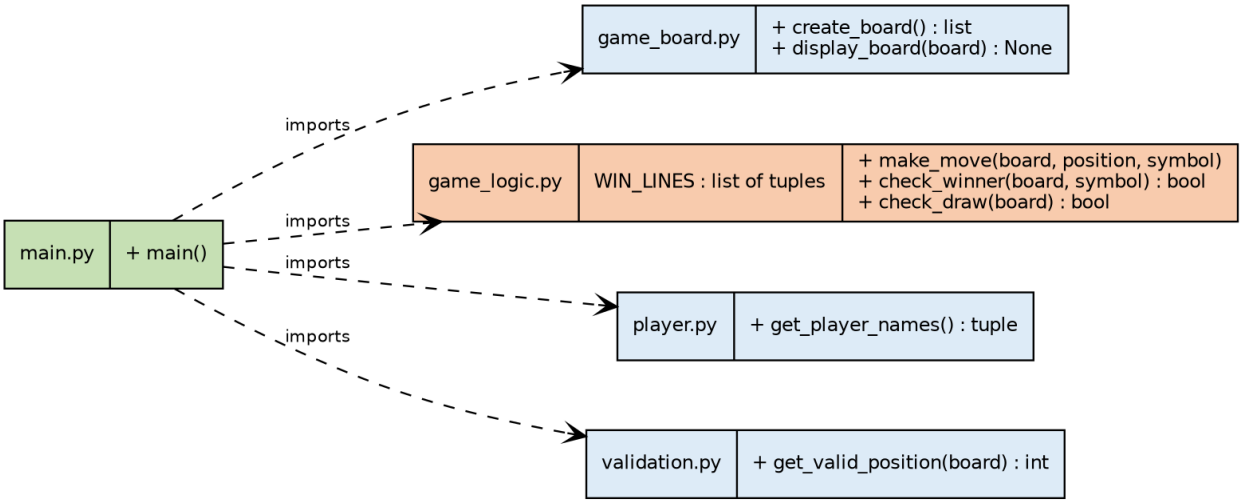


Figure 5: Component diagram showing modules and their functions

6.5 ER Diagram / Database Design

Not applicable. The game does not use a database or any file storage; all state exists only in memory while the program runs. The only data structure is the board, described below.

6.6 Board Data Design

The board is a one-dimensional list of 9 strings. An empty cell holds a single space and an occupied cell holds 'X' or 'O'. Index i corresponds to the position number i + 1 shown to the players.

Position shown	1	2	3	4	5	6	7	8	9
List index	0	1	2	3	4	5	6	7	8

7. Design Decisions and Rationale

Decision	Alternatives considered	Rationale
Store the board as a flat list of 9 cells	2D list (3 x 3)	A flat list makes the position number map directly to an index, simplifies validation and printing, and makes the winning lines simple index triples.
Keep all 8 winning lines in a WIN_LINES constant	Long chain of if-statements	Data-driven checking is shorter, easier to verify and cannot miss a line by copy-paste error. A single any/all expression checks every line.
Separate validation.py from game logic	Validate inside main loop	Input rules can change (for example a GUI) without touching the game rules, and the validator can be unit tested with mocked input.
Convert to 0-based index inside the validator	Convert in main.py	The rest of the program only ever sees a safe list index, so no other module has to remember the off-by-one adjustment.
Check for a win before checking for a draw	Draw first	A move that fills the last cell may also win; checking the win first ensures such a game is not wrongly reported as a draw.
Default names for blank input	Force non-empty name	Faster to start playing and no additional error case.
Console interface using only the standard library	Tkinter / Pygame GUI	Keeps the project portable and focused on logic and design; a GUI can be added later on top of the same modules.
Pass board as a parameter (no global variable)	Global board	Functions stay pure and easy to test; no hidden shared state.

8. Implementation Details

8.1 Project Structure

```
tic-tac-toe/
|-- main.py          # Game loop and entry point
|-- game_board.py    # create_board(), display_board()
|-- game_logic.py    # make_move(), check_winner(), check_draw()
|-- player.py        # get_player_names()
|-- validation.py    # get_valid_position()
|-- test_game.py     # 15 unit tests
|-- README.md        # Overview, install, run, test instructions
`-- statement.md     # Problem statement, scope, target users
```

8.2 game_board.py

create_board() returns a new list of nine spaces. display_board() builds a list of printable cells in which each empty cell is replaced by its position number (index + 1), then prints the grid with separators.

```
def create_board():
    return [" "] * 9

def display_board(board):
    cells = [c if c != " " else str(i + 1) for i, c in enumerate(board)]
    print()
    print(f" {cells[0]} | {cells[1]} | {cells[2]} ")
    print("---+---+---")
    ...
```

8.3 game_logic.py

The eight winning lines are stored once as index triples. check_winner() returns True if, for any line, all three cells contain the given symbol. check_draw() returns True when no empty cell is left. main.py calls check_draw() only after check_winner() has returned False.

```
WIN_LINES = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8),  # rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8),  # columns
    (0, 4, 8), (2, 4, 6),             # diagonals
]

def make_move(board, position, symbol):
    board[position] = symbol

def check_winner(board, symbol):
    return any(all(board[i] == symbol for i in line) for line in WIN_LINES)

def check_draw(board):
    return " " not in board
```

8.4 validation.py

get_valid_position() loops until it receives acceptable input. It first checks that the text consists of digits, then that the number is within 1-9, and finally that the chosen cell is empty. Each failure prints its own message. On success it returns the 0-based index.

```
def get_valid_position(board):
    while True:
        choice = input("Choose a position (1-9): ").strip()
        if not choice.isdigit():
            print("Please enter a number.")
            continue
        position = int(choice) - 1
        if not 0 <= position <= 8:
            print("Number must be between 1 and 9.")
        elif board[position] != " ":
            print("That spot is already taken.")
        else:
            return position
```

8.5 player.py

```
def get_player_names():
    name1 = input("Enter name for Player 1 (X): ").strip() or "Player 1"
    name2 = input("Enter name for Player 2 (O): ").strip() or "Player 2"
    return name1, name2
```

8.6 main.py - Game Loop

main() prints the banner, reads the names, creates the board and starts with Player 1 and the symbol X. Inside an endless loop it displays the board, obtains a valid position, places the move, and then checks the result. When a win or draw is found the loop is left with break; otherwise the current player and symbol are swapped.

```
while True:
    display_board(board)
    print(f"{current_player}'s turn ({current_symbol})")
    position = get_valid_position(board)
    make_move(board, position, current_symbol)
    if check_winner(board, current_symbol):
        display_board(board); print(f"Congratulations! {current_player} wins!"); break
    if check_draw(board):
        display_board(board); print("It's a draw!"); break
    # switch player (X <-> O)
```

8.7 Complexity

Board size is fixed at 9 cells, so every operation is effectively constant time: a win check performs at most 24 comparisons and a game lasts at most 9 turns. Memory use is a single list of 9 short strings.

8.8 Version Control

The project is maintained in a Git repository on GitHub with the source files, tests, README.md and statement.md. Development was done in small commits, one per module or feature (for example board module, game logic, validation, tests, documentation).

9. Screenshots / Results

The screens below are the actual terminal output of the program (captured by running main.py with the inputs shown after each prompt).

9.1 Scenario A: A win with invalid inputs

Rahul (X) and Aman (O) play. Rahul first types 'abc' (rejected), Aman tries the occupied cell 5 (rejected) and then 10 (rejected). Rahul finally wins on the diagonal 3-5-7.

```
================================
        TIC-TAC-TOE GAME
================================
Enter name for Player 1 (X): Rahul
Enter name for Player 2 (O): Aman

 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9

Rahul's turn (X)
Choose a position (1-9): abc
Please enter a number.
Choose a position (1-9): 5

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Aman's turn (O)
Choose a position (1-9): 5
That spot is already taken.
Choose a position (1-9): 10
Number must be between 1 and 9.
Choose a position (1-9): 1

 O | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Rahul's turn (X)
Choose a position (1-9): 3

 O | 2 | X
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Aman's turn (O)
Choose a position (1-9): 2

 O | O | X
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Rahul's turn (X)
Choose a position (1-9): 7

 O | O | X
---+---+---
 4 | X | 6
---+---+---
 X | 8 | 9

Congratulations! Rahul wins!
```

9.2 Scenario B: A draw

Both names are left blank so the defaults are used. The nine moves 1, 2, 3, 5, 4, 6, 8, 7, 9 fill the board without any line being completed, so the game reports a draw. Only the end of the game is shown.

```
[... first seven moves omitted ...]
Player 2's turn (O)
Choose a position (1-9): 7

 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | 9

Player 1's turn (X)
Choose a position (1-9): 9

 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | X

It's a draw!
```

10. Testing Approach

Testing combines automated unit tests for every module with manual end-to-end play-throughs. Functions that read input are tested by replacing the built-in input() with the unittest.mock.patch tool, and functions that print are tested by capturing standard output.

10.1 How to run the tests

```
python -m unittest -v test_game.py
```

10.2 Unit test cases (test_game.py)

Module	Test case	Expected behaviour	Result
game_board	test_create_board_is_empty	New board has nine empty cells	Pass
game_board	test_display_shows_numbers_for_empty_cells	Empty cells printed as 1-9	Pass
game_board	test_display_shows_symbols	Occupied cell shows X/O instead of number	Pass
game_logic	test_make_move_places_symbol	Symbol stored at the chosen index	Pass
game_logic	test_all_eight_winning_lines_detected	Each of 8 lines is recognised as a win for its owner only	Pass
game_logic	test_no_winner_on_empty_board	No win on an empty board	Pass
game_logic	test_draw_when_board_full	Full board with no line is a draw and has no winner	Pass
game_logic	test_not_draw_when_cells_left	Board with empty cells is not a draw	Pass
player	test_names_are_read_and_trimmed	Names returned with spaces removed	Pass
player	test_blank_names_use_defaults	Blank input gives Player 1 / Player 2	Pass
validation	test_valid_input_returns_zero_based_index	Input 5 returns index 4	Pass
validation	test_rejects_non_numeric_then_accepts	'abc' and empty text rejected, then 3 accepted	Pass
validation	test_rejects_out_of_range	0 and 10 rejected, 9 accepted	Pass
validation	test_rejects_taken_cell	Occupied cell rejected, free cell accepted	Pass
validation	test_negative_number_rejected	'-1' rejected, then 4 accepted	Pass

10.3 Test run output

```
----------------------------------------------------------------------
Ran 15 tests in 0.002s
OK
```

All 15 tests pass. The tests cover all four logic and input modules; main.py is verified through the manual scenarios below.

10.4 Manual end-to-end tests

No.	Scenario	Input	Expected	Actual
M1	Win on a diagonal	5, 1, 3, 2, 7	Player 1 wins	'Congratulations! Rahul wins!'
M2	Draw	1, 2, 3, 5, 4, 6, 8, 7, 9	Draw message	'It's a draw!'
M3	Text instead of number	abc	Error, ask again	'Please enter a number.'
M4	Occupied cell	5 (twice)	Error, ask again	'That spot is already taken.'
M5	Number out of range	10	Error, ask again	'Number must be between 1 and 9.'
M6	Blank player names	Enter, Enter	Default names used	'Player 1' and 'Player 2' shown

11. Challenges Faced

- **Off-by-one errors:** Players see positions 1-9 while Python lists start at 0. This was solved by converting once, inside the validator, and documenting it.
- **Correct win detection:** Writing eight separate conditions is error-prone. Moving the lines into the WIN_LINES data structure made the check compact and easy to test for all eight cases.
- **Order of checks:** If the draw were checked first, a winning last move would be reported as a draw. The win check is therefore always done first.
- **Robust input handling:** Players may type letters, blanks, zero, negative numbers or occupied cells. Each case needed its own clear message inside one loop.
- **Testing code that uses input() and print():** Solved with unittest.mock.patch and captured standard output.

11.1 Known limitations

- Pressing Ctrl+C or Ctrl+D (end of input) ends the program with a Python traceback instead of a friendly exit message.
- isdigit() also accepts some special Unicode digits such as the superscript two; converting these with int() raises an error. Using isdecimal() instead would remove this edge case.
- The game runs a single round per launch and does not keep score.

12. Learnings and Key Takeaways

- Splitting a program into modules with one responsibility each makes it easier to understand, test and change.
- Storing rules as data (WIN_LINES) is usually simpler and safer than long chains of conditions.
- Validating input in a dedicated loop makes a program robust; user input should never be trusted.
- Functions that receive their data as parameters (instead of using globals) are far easier to unit test.
- Diagrams (use case, flowchart, sequence, component) help plan the design before coding and explain it afterwards.
- Automated tests give confidence when the code is modified later.

13. Future Enhancements

- **Computer opponent:** random moves first, later the minimax algorithm for an unbeatable AI.
- **Replay and scoreboard:** ask 'Play again?' and keep win/draw counts across rounds, optionally saved to a file.
- **Graphical interface:** Tkinter or Pygame front end reusing game_logic.py unchanged.
- **Graceful exit handling:** catch KeyboardInterrupt and EOFError, and allow typing 'q' to quit.
- **Larger boards:** configurable board size (4 x 4, 5 x 5) with generated winning lines.
- **Logging and continuous testing:** game log for debugging and a GitHub Actions workflow to run the tests automatically.